ReconForge Severity Classification Criteria

Author: Andrews Ferreira
Version: 2.0
Purpose: Define strict evidence requirements for finding severity and confidence levels, reducing false positives and ensuring credibility in real penetration tests.

Classification Overview

ReconForge uses a **two-dimensional classification system**:

1. **Severity** – Impact level of the finding (critical → info)
2. **Confidence** – Strength of evidence supporting the finding (confirmed → heuristic)

Enforcement Rule

The `FindingsManager` enforces strict clamping: **severity is automatically capped based on confidence level**. This prevents weak signals from being reported as high-severity vulnerabilities.

Confidence Level	Maximum Allowed Severity	Description
`confirmed`	`critical`	Exploited or verified – no cap
`high`	`critical`	Strong tool-validated evidence – no cap
`medium`	`high`	Moderate evidence – capped below critical
`low`	`medium`	Weak evidence – capped at medium
`heuristic`	`low`	Pattern-only detection – capped at low

Example: A finding with `severity="high"` and `confidence="heuristic"` will be automatically clamped to `severity="low"`.

Severity Levels

🔴 Critical

Requirements:
- Confirmed exploitability with proof-of-concept

- High impact (RCE, authentication bypass, data exfiltration)
- Tool-verified (e.g., sqlmap confirms injection, Nuclei critical template match)

Examples:
- SQL injection confirmed by sqlmap with data extraction
- JWT `alg: none` allowing token forgery
- RCE via deserialization with command execution proof

NOT Critical:
- Missing security headers
- Service version detection
- Parameter name patterns

🟠 High

Requirements:
- Strong evidence of exploitability (error messages, timing differences)
- Significant impact potential
- Requires at least `medium` confidence

Examples:
- Nuclei high-severity template match with extracted data
- WordPress plugin with known CVE and confirmed version
- Exposed API specification with sensitive endpoints (verified accessible)

NOT High:
- URL patterns matching `/admin` or `/users`
- IDOR-prone parameter names (e.g., `user_id`)
- HTTP 500 errors from fuzzing
- Service presence on a port

🟡 Medium

Requirements:
- Moderate evidence requiring further validation
- Potential for exploitation if confirmed
- Requires at least `low` confidence

Examples:
- API key passed in query parameter (confirmed in spec)
- Admin endpoint returning HTTP 200 (needs functionality verification)
- Multiple API versions detected (deprecation concern)

NOT Medium:
- Missing `X-Frame-Options` header
- Server header disclosure
- Technology fingerprint detection

🔵 Low

Requirements:
- Weak evidence or minimal impact
- May be informational with slight security relevance
- Allowed for any confidence level

Examples:
- Heuristic IDOR candidates (URL/parameter patterns)
- Server errors triggered by fuzzing (no injection evidence)
- Rate limiting headers absent
- X-Powered-By header exposure

⚪ Info

Requirements:
- No direct security impact
- Informational observation useful for context
- Always allowed

Examples:
- Technology detection (WordPress, Apache, nginx)
- Open port discovery
- Missing security headers
- Server header disclosure
- Service version information
- JWT using HS256 (valid algorithm)
- API version detection

Confidence Levels

confirmed

- Finding verified through exploitation or direct observation
- Tool output unambiguously proves the issue
- **Examples:** sqlmap confirms injection, WPScan confirms vulnerable plugin version, `alg: none` JWT

high

- Tool output strongly indicates the issue
- Multiple corroborating signals
- **Examples:** Nuclei template match, version-confirmed CVE, authenticated scan result

medium

- Single tool signal requiring validation
- Moderate certainty but not conclusive
- **Examples:** HTTP probe response, API spec analysis, single Nikto finding

low

- Circumstantial evidence
- Requires significant manual verification
- **Examples:** Status code anomalies, response size differences, timing variations

heuristic

- **Pattern-based detection only – NO concrete evidence**
- URL patterns, parameter names, endpoint naming conventions
- These are investigation suggestions, NOT vulnerabilities

- **Examples:** URL contains `/users/`, parameter named `user_id`, endpoint path matches `/admin`

What is NOT a Vulnerability

These patterns **must never be classified as high or critical**:

Pattern	Why It's Not a Vulnerability	Correct Classification
Parameter names (`id`, `user_id`)	Standard API design	`info` / `heuristic`
Endpoint patterns (`/admin` , `/api/v1/users`)	Normal URL structure	`info` / `heuristic`
HTTP 500 errors	Normal error handling	`low` / `heuristic`
Missing security headers	Defence-in-depth only	`info` / `confirmed`
JWT using HS256	Valid standard algorithm	`info` / `confirmed`
Presence of parameters	Normal API functionality	`info` / `heuristic`
Service running on a port	Normal network operation	`info` / `confirmed`
Version detection without CVE	Information gathering	`info` / `confirmed`

Before/After Examples

1. BOLA/IDOR Pattern Detection (API Module)

BEFORE:

```
severity: high, confidence: low
type: vulnerability
description: "Potential BOLA/IDOR endpoint: /api/users/123"
evidence: "URL contains resource identifier pattern: /users/"
```

AFTER:

```
severity: low, confidence: heuristic
type: information
description: "BOLA/IDOR test candidate (heuristic): /api/users/123"
evidence: "URL contains resource identifier pattern '/users/' — requires manual
verification with multiple user contexts"
```

2. JWT HS256 Algorithm (API Module)

BEFORE:

```
severity: low, confidence: medium
type: misconfiguration
description: "JWT uses symmetric algorithm (HS256)"
```

AFTER:

```
severity: info, confidence: confirmed
type: information
description: "JWT uses symmetric algorithm (HS256) — this is a valid configuration"
```

3. Missing Security Headers (Web Module)

BEFORE:

```
severity: low, confidence: confirmed
type: misconfiguration
description: "Missing X-Frame-Options header (clickjacking risk)"
```

AFTER:

```
severity: info, confidence: confirmed
type: information
description: "Missing X-Frame-Options header (clickjacking defence-in-depth)"
```

4. HTTP 500 from Fuzzing (API Module)

BEFORE:

```
severity: medium, confidence: medium
type: vulnerability
description: "Server error triggered by fuzzing: /api/endpoint"
evidence: "HTTP 500 with input 'test'"
```

AFTER:

```
severity: low, confidence: heuristic
type: information
description: "Server error triggered by fuzzing (heuristic): /api/endpoint"
evidence: "HTTP 500 with input 'test' — requires manual analysis of response body for
injection evidence"
```

5. Service Detection (Surface Module)

BEFORE:

```
severity: high, confidence: medium
type: attack_vector
description: "Attack vector: smb on port 445 — EternalBlue, null sessions, relay at-
tacks"
```

AFTER:

```
severity: info, confidence: confirmed
type: information
description: "Service of interest: smb on port 445 — Investigate: null sessions,
signing, known CVEs"
```

6. Admin Endpoint (API Module)

BEFORE:

```
severity: high, confidence: medium
type: vulnerability
description: "Administrative/internal endpoint accessible: /api/admin"
```

AFTER:

```
severity: medium, confidence: medium
type: exposure
description: "Administrative/internal endpoint accessible: /api/admin"
evidence: "HTTP 200 response on admin-pattern URL (verify actual functionality manu-
ally)"
```

Adding New Findings – Checklist

When adding a new finding to any module, answer these questions:

1. **Is there concrete evidence?** (tool output, response content, timing proof)
   - If NO → confidence is `heuristic` or `low`
2. **Does the evidence prove exploitability?**
   - If NO → severity ≤ `medium`
3. **Is this based on pattern matching only?** (URL patterns, parameter names)
   - If YES → confidence = `heuristic`, severity ≤ `low`
4. **Is this a defence-in-depth observation?** (missing headers, version info)
   - If YES → severity = `info`
5. **Was this confirmed by a specialised tool?** (sqlmap, Nuclei, WPScan)
   - If YES → confidence ≥ `high`
6. **Can the FindingsManager clamping be relied upon?**
   - YES – always set the correct confidence level and let the system enforce caps

---

Implementation Details

The `FindingsManager._clamp_severity()` function automatically enforces these rules:

```
# Confidence → Maximum severity mapping
_CONFIDENCE_SEVERITY_CAP = {
    "confirmed": "critical",   # No cap
    "high":      "critical",   # No cap
    "medium":    "high",       # Medium confidence → max high
    "low":       "medium",     # Low confidence → max medium
    "heuristic": "low",        # Heuristic → max low
}
```

When severity is clamped, the finding description is prefixed with `[severity clamped: original→clamped]` to maintain transparency.

The `FindingsManager.clamped_count` property tracks how many findings were affected, and the markdown report includes a confidence breakdown section for audit purposes.